Paired-mutation obligation (contract row 9 / C-005) — documented, not implemented here

Contract row 9 (`contracts/catch-set-comparison-harness.md` RED fixtures table, kind "self"): "invert CS-004's comparison ⇒ `cs_bad_dropped_gate` passes; meta-test must catch."

Per `contracts/common-conventions.md` C-005: "Each tool also ships a paired §1.1 mutation of its own detection logic that flips a golden-bad to exit 0; the meta-test asserts the mutation is caught."

This is T064's obligation, not T050's. T050 (this RED test / this fixture set) exists specifically because `constitution/scripts/fastcycle/ gates/catchset_compare.py` does not exist yet (see the main test's absence-proof section) -- there is no detection logic to mutate. Fabricating a premature mutation-test for code that does not exist would itself be a §11.4.6 violation (asserting a fact about an artefact's behaviour with nothing to observe).

What the meta-test T064 must ship (named precisely, so this obligation is traceable and not lost per §11.4.197): T064's implementation of `catchset_compare.py compare` MUST have a paired `scripts/testing/meta_test_false_positive_proof.sh` mutation that inverts CS-004's superset condition -- for example, changing the check from "violated iff ANY row has `old=CAUGHT ∧ new=MISSED`" to a weaker or inverted form (e.g. requiring ALL rows to be `old=CAUGHT ∧ new=MISSED` before reporting a violation, or flipping the `∧` to `∨`) -- and re-running it against `cs_bad_dropped_gate/` (this directory's own fixture, whose real `expected.json` says `expected_exit_code: 1`) MUST observe the mutated tool wrongly report `exit 0` (superset holds) on that fixture.

The meta-test then asserts that mutation is caught (i.e., some OTHER check -- golden-fixture regression, or the mutation-detection harness itself -- FAILs when that inversion is applied), closing the loop C-005 requires.

This file is the durable record of that obligation so it is not silently dropped between T050 (RED, this task) and T064 (GREEN implementation).